

1. 개요

미국 MIT에서 교육용 운영체제로 개발한 xv6(x86, multiprocessor, Unix V6 재구현) 플랫폼 상에서 “Hello xv6 World”를 출력하는 helloxv6와 프로세스의 상태를 출력하는 “psinfo” 응용 프로그램을 구현하였다. 이 프로그램을 위해서 시스템 콜 추가 작업을 수행했다.

helloxv6 프로그램의 경우 커널의 시스템 콜(sys_hello_number)을 통해 “Hello, xv6! Your number is n”을 출력하고, 커널로부터 반환값을 받아 유저 공간에서 결과를 출력한다.

psinfo 프로그램의 경우 특정 PID(혹은 자기 자신)의 프로세스 정보를 커널에서 수집하여 유저 공간으로 복사받아 출력한다 추가한 시스템 콜은 2개이다.(int hello_number(int n);, int get_procinfo(int pid, struct procinfo *uinfo);)

구현 과정에서 커널/유저 경계, 시스템 콜 경로, 동기화(스핀락), 커널 구조체 레이아웃과 사용자 구조체 레이아웃의 일치 개념에 대해 어느정도 이해할 수 있었다. 또한 xv6의 전역 프로세스 테이블인 ptable을 안전하게 접근하기 위해 스핀락을 사용하고, copyout()으로 커널 -> 유저 공간으로 메모리 복사를 수행하는 과정에서 이슈가 있었는데 이를 해결하면서 더 이해할 수 있었다

2. 상세 설계

(1) xv6 설치 과정

\$ git clone <https://github.com/mit-pdos/xv6-public> 실행 결과

```
seoknam@seoknam-VirtualBox:~/OS$ git clone https://github.com/mit-pdos/xv6-public
'xv6-public'에 복제합니다...
remote: Enumerating objects: 13990, done.
remote: Total 13990 (delta 0), reused 0 (delta 0), pack-reused 13990 (from 1)
오브젝트를 받는 중: 100% (13990/13990), 17.24 MiB | 2.49 MiB/s, 완료.
델타를 알아내는 중: 100% (9466/9466), 완료.
seoknam@seoknam-VirtualBox:~/OS$ ls
OS_1  xv6-public
```

\$ apt-get install qemu-kvm

```
seoknam@seoknam-VirtualBox:~/OS$ sudo apt-get install qemu-kvm
패키지 목록을 읽는 중입니다... 완료
의존성 트리를 만드는 중입니다... 완료
상태 정보를 읽는 중입니다... 완료
주의, 'qemu-kvm' 대신에 'qemu-system-x86' 패키지를 선택합니다
패키지 qemu-system-x86는 이미 최신 버전입니다 (1:6.2+dfsg-2ubuntu6.26).
0개 업그레이드, 0개 새로 설치, 0개 제거 및 196개 업그레이드 안 함.
```

\$make

\$make qemu-nox

make qemu-nox 실행 결과

```
SeaBIOS (version 1.15.0-1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B4A0+1FECB4A0 CA00

Booting from Hard Disk...
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ █
```

ls 실행 결과

```
SeaBIOS (version 1.15.0-1)

lpxe (https://lpxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B4A0+1FECB4A0 CA00

Booting from Hard Disk...
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ ls
.          1 1 512
..         1 1 512
README    2 2 2286
cat        2 3 15544
echo       2 4 14424
forktest   2 5 8860
grep       2 6 18380
init       2 7 15048
kill       2 8 14508
ln         2 9 14408
ls         2 10 16976
mkdir     2 11 14532
rm         2 12 14516
sh         2 13 28564
stressfs   2 14 15440
usertests  2 15 62936
wc         2 16 15960
zombie     2 17 14092
helloxv6   2 18 14508
psinfo     2 19 15116
console    3 20 0
$
```

(2) 수정/추가한 파일 및 핵심 변경점

user.h: hello_number, struct procinfo, get_procinfo 프로토타입 추가

```
27 int hello_number(int n);

42 // 유저용 선언
43 struct procinfo{
44     int pid;        // 대상 PID
45     int ppid;       // 부모 PID
46     int state;      // enum procstate 값
47     uint sz;        // 메모리 크기(바이트)
48     char name[16];  // 프로세스 이름
49 };
50 int get_procinfo(int pid, struct procinfo *uinfo);
```

syscall.h: SYS_hello_number, SYS_get_procinfo 번호 추가

```
23 #define SYS_hello_number 22
24 #define SYS_get_procinfo 23
```

usys.S: SYSCALL(hello_number), SYSCALL(get_procinfo) 추가

```
32 SYSCALL(hello_number)
33 SYSCALL(get_procinfo)
```

syscall.c: extern int sys_hello_number(void); extern int sys_get_procinfo(void);, syscalls[] 테이블에 인덱스 매핑 추가

```
106 extern int sys_hello_number(void); // sys_hello_number 시스템콜 추가
107 extern int sys_get_procinfo(void); // sys_get_procinfo 시스템콜 추가

131 [SYS_hello_number] sys_hello_number, // sys_hello_number 시스템콜 추가
132 [SYS_get_procinfo] sys_get_procinfo, // sys_get_procinfo 시스템콜 추가
```

sysproc.c: sys_hello_number() 및 sys_get_procinfo() 구현 (아래 각 시스템 콜 호출 과정에서 구현한 코드 설명)

proc.h/proc.c: struct ptable_t 타입 정의(이름 있는 구조체), extern/정의 통일

```
65 // ptable을 sysproc.c에서 쓰기 위해 extern으로 선언
66 // 이때 익명 struct를 쓰지 않고, 공용 타입을 만들어서 extern/define을 맞춰줌
67
68 struct ptable_t {
69     struct spinlock lock;
70     struct proc proc[NPROC];
71 };
72
73 extern struct ptable_t ptable;
```

(proc.h)

```

10 //struct {
11 //  struct spinlock lock;
12 //  struct proc proc[NPROC];
13 //} ptable;
14
15 struct ptable_t ptable;

```

(proc.c)

spinlock.h: include guard 추가(컴파일 에러를 막기 위함).

include guard는 동일한 헤더가 여러 번 include 되어도 컴파일 에러를 방지하는 것이다.

그리고 struct cpu를 구조체 보다 위에서 전방선언 해주었는데, 이렇게 되면 struct cpu의 실제 정의를 몰라도 struct cpu*로 포인터를 선언 가능해진다. 이러면 spinlock.h를 cpu.h에 의존하지 않게 만들어 헤더 간 의존성을 줄이고, include 순서 문제를 방지할 수 있다.

```

1 #ifndef SPINLOCK_H
2 #define SPINLOCK_H
3
4 #include "types.h"
5
6 struct cpu; // 전방선언
7
8 // Mutual exclusion lock.
9 struct spinlock {
10  uint locked; // 현재 이 락이 잡혀있으면 1, 아니면 0
11
12  char *name; // 디버깅용 문자열 이름. 예를 들어 "ptable" 같은 이름을 붙여두면, 프로세스 테이블 락인지 바로 알 수 있음
13  struct cpu *cpu; // 현재 이 락을 들고 있는 cpu를 가리킨다.
14  uint pcs[10]; // 락을 잡았을 때의 호출 스택(program counter)들을 저장한다
15
16 };
17
18 #endif

```

Makefile: UPROGS에 _helloxv6, _psinfo 추가

: UPROGS에 나열된 것들은 xv6 유저 프로그램으로 컴파일되어, xv6 파일시스템 이미지에 포함된다.

앞의 _는 Makefile에서 규칙을 찾기 위한 접두사이며, 예를 들어 _psinfo를 등록한 것은, 실제로는 psinfo.c를 컴파일해 psinfo를 실행 파일로 생성한다는 의미이다. 만약 UPROGS에 없으면, psinfo.c를 작성해도 빌드 결과물로 xv6 파일시스템에 포함되지 않는다.

```

168 UPROGS=\
169  _cat\
170  _echo\
171  _forktest\
172  _grep\
173  _init\
174  _kill\
175  _ln\
176  _ls\
177  _mkdir\
178  _rm\
179  _sh\
180  _stressfs\
181  _usertests\
182  _wc\
183  _zombie\
184  _helloxv6\
185  _psinfo\

```

(2) 구현한 함수 프로토타입

```

int hello_number(int n); // sys_hello_number() 호출을 위한 stub
int get_procinfo(int pid, struct procinfo *uinfo); // sys_get_procinfo() 호출을 위한 stub

```

```

//syscall.c
extern int sys_hello_number(void); // 유저 영역에서 hello_number 사용시 호출되는 커널 영역의 시스템 콜
extern int sys_get_procinfo(void); // 유저 영역에서 get_procinfo 사용시 호출되는 커널 영역의 시스템 콜

```

(3) hello_number 시스템 콜 추가 과정

hello_number 시스템 콜은 정수 하나를 인자로 받아 커널 내부에서 해당 정수를 메시지에 포함하여 화면에 표시하고, 연산 결과를 호출한 프로세스에 리턴하는 함수이다

(1) syscall.h 수정

SYS_hello_number를 22번째 시스템 콜로 추가한다. 기존 xv6는 시스템 콜이 21개 있다고 생각할 수 있다.

```

seoknam@seoknam-VirtualBox: ~/... x      syscall.h+ (-/OS/xv6-public) -VIM x
1 // System call numbers
2 #define SYS_fork    1
3 #define SYS_exit    2
4 #define SYS_wait    3
5 #define SYS_pipe    4
6 #define SYS_read    5
7 #define SYS_kill    6
8 #define SYS_exec    7
9 #define SYS_fstat    8
10 #define SYS_chdir   9
11 #define SYS_dup    10
12 #define SYS_getpid  11
13 #define SYS_sbrk    12
14 #define SYS_sleep   13
15 #define SYS_uptime  14
16 #define SYS_open    15
17 #define SYS_write   16
18 #define SYS_mknod   17
19 #define SYS_unlink  18
20 #define SYS_link    19
21 #define SYS_mkdir   20
22 #define SYS_close   21
23 #define SYS_hello_number 22

```

(2) sysproc.c 수정

: 커널에서 실제로 동작할 시스템 콜 핸들러 함수를 구현하는 것이다. 이때 argint() 함수를 이용해서 시스템 콜의 첫 번째 인자를 int로 꺼내서 변수 n에 저장한다. 마지막으로 n*2 값을 반환하면, 이 값이 사용자 프로그램으로 전달된다.

```

93 int
94 sys_hello_number(void){
95     int n;
96     if(argint(0, &n) < 0)
97         return -1;
98     cprintf("Hello, xv6! Your number is %d\n", n);
99     return n * 2;
100 }

```

(3) syscall.c 수정

시스템 콜 디스패처에 새로운 호출을 등록한다. 이로써 커널이 트랩을 통해 hello_number 시스템 콜 번호(22)를 받으면, sys_hello_number 함수를 호출하도록 연결된다.

```

104 extern int sys_write(void);
105 extern int sys_uptime(void);
106 extern int sys_hello_number(void);
107
108 static int (*syscalls[])(void) = {
109 [SYS_fork]    sys_fork,
110 [SYS_exit]    sys_exit,
111 [SYS_wait]    sys_wait,
112 [SYS_pipe]    sys_pipe,
113 [SYS_read]    sys_read,
114 [SYS_kill]    sys_kill,
115 [SYS_exec]    sys_exec,
116 [SYS_fstat]   sys_fstat,
117 [SYS_chdir]   sys_chdir,
118 [SYS_dup]     sys_dup,
119 [SYS_getpid]  sys_getpid,
120 [SYS_sbrk]    sys_sbrk,
121 [SYS_sleep]   sys_sleep,
122 [SYS_uptime]  sys_uptime,
123 [SYS_open]    sys_open,
124 [SYS_write]   sys_write,
125 [SYS_mknod]   sys_mknod,
126 [SYS_unlink]  sys_unlink,
127 [SYS_link]    sys_link,
128 [SYS_mkdir]   sys_mkdir,
129 [SYS_close]   sys_close,
130 [SYS_hello_number] sys_hello_number,
131 };
132

```

(4) usys.S 수정

사용자 영역에서 시스템 콜을 부를 수 있도록 어셈블리 코드를 추가한다. .S로 끝나는 파일들은 어셈블리 코드 파일이다. usys.S 파일을 SYSCALL(name) 형식의 매크로를 이용해 각 시스템 콜의 유저 공간에서의 함수를 정의하고 있다. 마지막 라인에 새로운 시스템 콜 이름을 넣어서 SYSCALL(hello_number)라고 써 주었다.

```

seoknam@seoknam-VirtualBox: ~/...  usys.S + (~/OS/xv6-public) - VIM
1 #include "syscall.h"
2 #include "traps.h"
3
4 #define SYSCALL(name) \
5     .globl name; \
6     name: \
7         movl $SYS_ ## name, %eax; \
8         int $T_SYSCALL; \
9         ret
10
11 SYSCALL(fork)
12 SYSCALL(exit)
13 SYSCALL(wait)
14 SYSCALL(pipe)
15 SYSCALL(read)
16 SYSCALL(write)
17 SYSCALL(close)
18 SYSCALL(kill)
19 SYSCALL(exec)
20 SYSCALL(open)
21 SYSCALL(mknod)
22 SYSCALL(unlink)
23 SYSCALL(fstat)
24 SYSCALL(link)
25 SYSCALL(mkdir)
26 SYSCALL(chdir)
27 SYSCALL(dup)
28 SYSCALL(getpid)
29 SYSCALL(sbrk)
30 SYSCALL(sleep)
31 SYSCALL(uptime)
32 SYSCALL(hello_number)

```

(5) user.h 수정

user.h에 hello_number() 함수에 대한 선언이 있어야 helloxy6.c를 컴파일할 때 링커가 extern 함수를 시스템 콜로 인식함.
즉 여기에도 선언이 있어야 시스템 콜을 쓸 수 있다.

```

seoknam@seoknam-VirtualBox: ~/...  user.h + (~/OS/xv6-public) - VIM
1 struct stat;
2 struct rtcdate;
3
4 // system calls
5 int fork(void);
6 int exit(void) __attribute__((noreturn));
7 int wait(void);
8 int pipe(int*);
9 int write(int, const void*, int);
10 int read(int, void*, int);
11 int close(int);
12 int kill(int);
13 int exec(char*, char**);
14 int open(const char*, int);
15 int mknod(const char*, short, short);
16 int unlink(const char*);
17 int fstat(int fd, struct stat*);
18 int link(const char*, const char*);
19 int mkdir(const char*);
20 int chdir(const char*);
21 int dup(int);
22 int getpid(void);
23 char* sbrk(int);
24 int sleep(int);
25 int uptime(void);
26 int hello_number(int n);
27

```

(4) get_procinfo 시스템 콜 추가 과정

(1) user.h와 sysproc.c에 아래의 동일한 구조체를 정의

```

42 // 유저용 선언
43 struct procinfo{
44     int pid;           // 대상 PID
45     int ppid;          // 부모 PID
46     int state;         // enum procstate 값
47     uint sz;           // 메모리 크기(바이트)
48     char name[16];     // 프로세스 이름
49 };
50 int get_procinfo(int pid, struct procinfo *uinfo);

```

(2) user.h에 get_procinfo() 함수 프로토타입 선언, procinfo 구조체 추가


```

43 // 유저용 선언
44 struct procinfo{
45     int pid;        // 대상 PID
46     int ppid;       // 부모 PID
47     int state;       // enum procstate 값
48     uint sz;         // 메모리 크기(바이트)
49     char name[16];   // 프로세스 이름
50 };
51 int get_procinfo(int pid, struct procinfo *uinfo);

```

(3) syscall.h에 SYS_get_procinfo을 define

```

22 #define SYS_close 21
23 #define SYS_hello_number 22
24 #define SYS_get_procinfo 23

```

(4) usys.S에 SYSCALL(get_procinfo) 추가

```

31 SYSCALL(uptime)
32 SYSCALL(hello_number)
33 SYSCALL(get_procinfo)

```

(5) syscall.c 수정(sys_get_procinfo(void) 선언 후, syscalls[] 배열에 등록)

```

105 extern int sys_uptime(void);
106 extern int sys_hello_number(void);
107 extern int sys_get_procinfo(void);

```

```

[SYS_get_procinfo] sys_get_procinfo, // sys_get_procinfo 시스템콜 추가
};

```

(6) sysproc.c에서 sys_get_procinfo(void){} 일부 구현

```

int sys_get_procinfo(void){
    int pid;
    char *uaddr;        // 유저 버퍼 시작 주소
    struct proc *p, *t;
    struct k_procinfo kinfo;

    // 시스템콜의 첫 번째 인자(0)를 int로 꺼내서 pid 변수에 저장
    // 사용자가 넣긴 pid값을 커널 변수 pid로 안전하게 가져온다
    if(argint(0, &pid) < 0)
        return -1;
    // 시스템콜의 두 번째 인자(1)를 포인터로 받아온다
    // 즉, 유저가 넣긴 포인터가 가리키는 주소(struct k_procinfo *uinfo의 주소)가 가리키는 주소를
    // uaddr에 저장한다
    if(argptr(1, &uaddr, sizeof(struct k_procinfo)) < 0)
        return -1;

    // 프로세스 테이블(pstable)에 접근할 때 동시성 문제를 막기 위해 락을 잡음
    // 락 관련 문제 발생
    acquire(&pstable.lock);

    // pid <= 0 : 호출한 자기 자신 정보 조회
    if(pid <= 0)
        t = myproc(); // 현재 실행 중인 프로세스를 가져오는 함수
    // pid > 0 : 해당 PID 프로세스의 정보를 조회
    else{
        t = 0;
        for(p = pstable.proc; p < &pstable.proc[NPROC]; p++){
            if(p->pid == pid) {
                t = p;
                break;
            }
        }
    }

    // 해당 프로세스(t)가 없거나 UNUSED 상태면 에러 반환
    if(t == 0 || (t->state == UNUSED)) {
        release(&pstable.lock);
        return -1;
    }

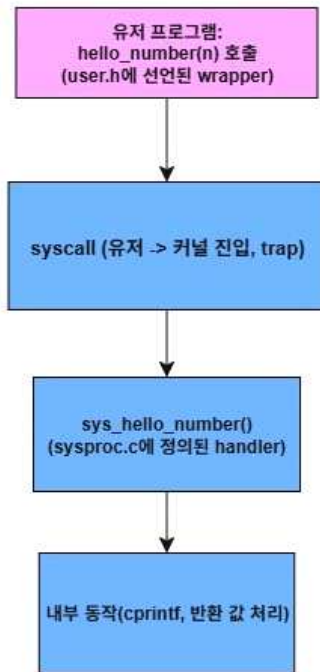
    // t에서 필요한 값 추출
    // 입력받은 PID의 프로세스 정보를 kinfo 구조체에 저장한다
    kinfo.pid = t->pid;                // 프로세스 PID
    kinfo.ppid = t->parent ? t->parent->pid : 0; // 부모 프로세스 PID(없으면 0)
    kinfo.state = t->state;             // 프로세스 상태
    kinfo.sz = t->sz;                  // 메모리 크기(bytes)
    safestrcpy(kinfo.name, t->name, sizeof(kinfo.name)); // 프로세스 이름

    // copyout/return을 하려면 락을 풀어야 한다
    release(&pstable.lock);

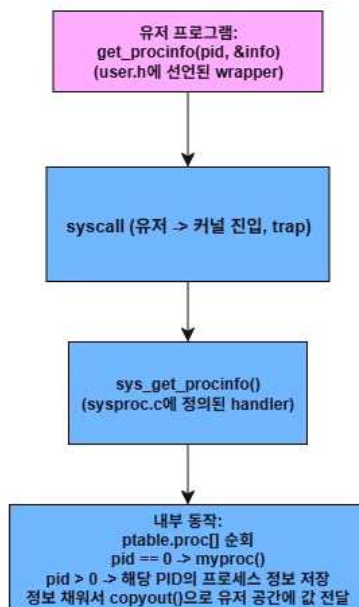
    // 유저 공간으로 복사
    // copyout()을 이용해 커널 공간의 kinfo를 사용자 버퍼(uaddr)로 복사한다.
    if(copyout(myproc()->pgdir, (uint)uaddr, (void*)&kinfo, sizeof(kinfo)) < 0)
        return -1;
    return 0;
}

```

(4) 기구화된 함수와 구현한 내용(함수) 간의 호출 그래프
 [hello_number 시스템 콜 관련 호출 그래프]



[get_procinfo 시스템 콜 관련 호출 그래프]



3. 결과

(1) 추가한 두 가지 시스템 콜(hello_number, get_procinfo)을 테스트하기 위한 사용자 프로그램 helloxv6.c와 psinfo.c의 내용 [helloxv6.c]

: 이 프로그램을 실행하면 커널의 hello_number 시스템 콜을 호출하고, 커널에서 출력한 메시지인 “Hello, xv6! Your number is n”를 확인한 후 시스템 콜이 리턴한 값을 사용자 프로그램 측에서 출력한다.

예를 들어 hello_number(5)에 대해서,

“Hello, xv6! Your number is 5”

“hello_number(5) returned 10” 이 출력되는 것이다

```

1 #include "types.h"
2 #include "stat.h"
3 #include "user.h"
4
5 int main(int argc, char *argv[]){
6     int res = hello_number(5);
7     printf(1, "hello_number(5) returned %d\n", res);
8     int res2 = hello_number(-7);
9     printf(1, "hello_number(-7) returned %d\n", res2);
10    int res3 = hello_number(0);
11    printf(1, "hello_number(0) returned %d\n", res3);
12    int res4 = hello_number(100000);
13    printf(1, "hello_number(100000) returned %d\n", res4);
14
15    exit();
16 }

```

[psinfo.c]

지정한 PID(또는 자기 자신)에 대한 핵심 프로세스 정보(pid, ppid, state, size, name)를 커널에 시스템 콜로 요청해서, 한 줄로 출력하는 코드이다.

실행 형식: psinfo [PID]이며, 인자를 주면 해당 PID의 정보를, 인자가 없으면 pid=0으로 간주하여 현재 프로세스(자기 자신) 정보를 출력한다. 커널에 있는 시스템콜 get_procinfo(pid, &info)를 호출해서 정보를 받아오고, 한 줄로 요약 출력한다.

```

1 //psinfo.c
2 #include "types.h"
3 #include "stat.h"
4 #include "user.h"
5
6 static char* s2str(int s){
7     // 상태 코드를 문자열로 변환한다
8     // 커널이 enum procstate[ ] 순서(정수 0~5)에 맞춰 문자열로 바꾼다.
9     switch(s){
10         case 0:
11             return "UNUSED";
12         case 1:
13             return "EMBRYO";
14         case 2:
15             return "SLEEPING";
16         case 3:
17             return "RUNNABLE";
18         case 4:
19             return "RUNNING";
20         case 5:
21             return "ZOMBIE";
22     }
23     return "UNKNOWN";
24 }
25
26 int main(int argc, char *argv[])
27 {
28     struct procinfo info;
29     int pid = (argc >= 2) ? atoi(argv[1]) : 0;    // pid=0이면 자기 자신
30
31     // info 구조체에 아래의 5가지 정보를 저장한다
32     // 이때 sysproc.c에서 정의했던 get_procinfo() 함수를 사용하면 된다.
33     if(get_procinfo(pid, &info) < 0)
34     {
35         // 시스템 콜 호출이 실패하면 에러 메시지 후 종료
36         // printf의 첫 인자 1은 xv6에서 stdout 파일 디스크립터이다.
37         printf(1, "psinfo: failed (pid=%d)\n", pid);
38         exit();
39     }
40     // 지정한 PID의 정보를 한 줄로 출력한다
41     printf(1, "PID=%d PPID=%d STATE=%s SZ=%d NAME=%s\n", info.pid, info.ppid, s2str(info.state), info.sz, info.name);
42     exit();
43 }

```

(2) 실행 결과

[helloxv6]


```

SeaBIOS (version 1.15.0-1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B4A0+1FECB4A0 CA00

Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ helloxv6
Hello, xv6! Your number is 5
hello_number(5) returned 10
Hello, xv6! Your number is -7
hello_number(-7) returned -14
Hello, xv6! Your number is 0
hello_number(0) returned 0
Hello, xv6! Your number is 100000
hello_number(100000) returned 200000
$

```

[psinfo]

```

SeaBIOS (version 1.15.0-1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B4A0+1FECB4A0 CA00

Booting from Hard Disk..xv6...
cpu0: starting 0
psb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ psinfo
PID=3 PPID=2 STATE=RUNNING SZ=12288 NAME=psinfo
$ psinfo 1
PID=1 PPID=0 STATE=SLEEPING SZ=12288 NAME=init
$ psinfo 9999
psinfo: failed (pid=9999)
$ psinfo 5
psinfo: failed (pid=5)
$ psinfo 2
PID=2 PPID=1 STATE=SLEEPING SZ=16384 NAME=sh
$ psinfo 3
psinfo: failed (pid=3)
$ psinfo 4
psinfo: failed (pid=4)
$ psinfo 6
psinfo: failed (pid=6)
$ psinfo 7
psinfo: failed (pid=7)
$

```

4. 주석 달린 소스코드

[proc.c]

// proc.c: 프로세스를 관리하는 코드

```

#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "x86.h"
#include "spinlock.h"
#include "proc.h"

```

```

// 원래 xv6는 익명 구조체를 바로 선언해서 ptable이라는 전역 변수를 만든다.
// 이 방식은 구조체에 이름이 없어서, ptable 말고는 그 타입을 재사용 할 수가 없다.
//struct {
//  struct spinlock lock;
//  struct proc proc[NPROC];
//} ptable;

```

```

// 그래서 타입 이름을 부여하는 방식으로 바꿨다.
// struct ptable_t라는 이름이 생겨, 필요하다면 다른 함수나 파일에서 타입을 참조할 수 있다
// ptable_t에 대한 정의는 proc.h에 있다
struct ptable_t ptable;

static struct proc *initproc;

int nextpid = 1;
extern void forkret(void);
extern void trapret(void);

static void wakeup1(void *chan);

// 프로세스 테이블에 대한 스핀락 초기화
// 동시성 문제 방지를 위해, 모든 프로세스 테이블 접근은 이 락을 잡고 해야 한다.
void
pinit(void)
{
    initlock(&ptable.lock, "ptable");
}

// Must be called with interrupts disabled
int
cpuid() {
    return mycpu() - cpus;
}

// Must be called with interrupts disabled to avoid the caller being
// rescheduled between reading lapicid and running through the loop.
struct cpu*
mycpu(void)
{
    int apicid, i;

    if(readeflags() & FL_IF)
        panic("mycpu called with interrupts enabled\n");

    apicid = lapicid();
    // APIC IDs are not guaranteed to be contiguous. Maybe we should have
    // a reverse map, or reserve a register to store &cpus[i].
    for (i = 0; i < ncpu; ++i) {
        if (cpus[i].apicid == apicid)
            return &cpus[i];
    }
    panic("unknown apicid\n");
}

// Disable interrupts so that we are not rescheduled
// while reading proc from the cpu structure
struct proc*
myproc(void) {
    struct cpu *c;
    struct proc *p;
    pushcli();
    c = mycpu();
    p = c->proc;
    popcli();
}

```

```

    return p;
}

//PAGEBREAK: 32
// Look in the process table for an UNUSED proc.
// If found, change state to EMBRYO and initialize
// state required to run in the kernel.
// Otherwise return 0.
static struct proc*
allocproc(void)
{
    struct proc *p;
    char *sp;

    acquire(&ptable.lock);

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        if(p->state == UNUSED)
            goto found;

    release(&ptable.lock);
    return 0;

found:
    p->state = EMBRYO;
    p->pid = nextpid++;

    release(&ptable.lock);

    // Allocate kernel stack.
    if((p->kstack = kalloc()) == 0){
        p->state = UNUSED;
        return 0;
    }
    sp = p->kstack + KSTACKSIZE;

    // Leave room for trap frame.
    sp -= sizeof *p->tf;
    p->tf = (struct trapframe*)sp;

    // Set up new context to start executing at forkret,
    // which returns to trapret.
    sp -= 4;
    *(uint*)sp = (uint)trapret;

    sp -= sizeof *p->context;
    p->context = (struct context*)sp;
    memset(p->context, 0, sizeof *p->context);
    p->context->eip = (uint)forkret;

    return p;
}

//PAGEBREAK: 32
// Set up first user process.
void
userinit(void)
{

```

```

struct proc *p;
extern char _binary_initcode_start[], _binary_initcode_size[];

p = allocproc();

initproc = p;
if((p->pgdir = setupkvm()) == 0)
    panic("userinit: out of memory?");
inituvm(p->pgdir, _binary_initcode_start, (int)_binary_initcode_size);
p->sz = PGSIZE;
memset(p->tf, 0, sizeof(*p->tf));
p->tf->cs = (SEG_UCODE << 3) | DPL_USER;
p->tf->ds = (SEG_UDATA << 3) | DPL_USER;
p->tf->es = p->tf->ds;
p->tf->ss = p->tf->ds;
p->tf->eflags = FL_IF;
p->tf->esp = PGSIZE;
p->tf->eip = 0; // beginning of initcode.S

safestrcpy(p->name, "initcode", sizeof(p->name));
p->cwd = namei("/");

// this assignment to p->state lets other cores
// run this process. the acquire forces the above
// writes to be visible, and the lock is also needed
// because the assignment might not be atomic.
acquire(&ptable.lock);

p->state = RUNNABLE;

release(&ptable.lock);
}

// Grow current process's memory by n bytes.
// Return 0 on success, -1 on failure.
int
growproc(int n)
{
    uint sz;
    struct proc *curproc = myproc();

    sz = curproc->sz;
    if(n > 0){
        if((sz = allocuvm(curproc->pgdir, sz, sz + n)) == 0)
            return -1;
    } else if(n < 0){
        if((sz = deallocuvm(curproc->pgdir, sz, sz + n)) == 0)
            return -1;
    }
    curproc->sz = sz;
    switchuvm(curproc);
    return 0;
}

// Create a new process copying p as the parent.
// Sets up stack to return as if from system call.
// Caller must set state of returned proc to RUNNABLE.
int

```

```

fork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *curproc = myproc();

    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }

    // Copy process state from proc.
    if((np->pgdir = copyuvm(curproc->pgdir, curproc->sz)) == 0){
        kfree(np->kstack);
        np->kstack = 0;
        np->state = UNUSED;
        return -1;
    }
    np->sz = curproc->sz;
    np->parent = curproc;
    *np->tf = *curproc->tf;

    // Clear %eax so that fork returns 0 in the child.
    np->tf->eax = 0;

    for(i = 0; i < NOFILE; i++)
        if(curproc->ofile[i])
            np->ofile[i] = filedup(curproc->ofile[i]);
    np->cwd = idup(curproc->cwd);

    safestrcpy(np->name, curproc->name, sizeof(curproc->name));

    pid = np->pid;

    acquire(&ptable.lock);

    np->state = RUNNABLE;

    release(&ptable.lock);

    return pid;
}

// Exit the current process. Does not return.
// An exited process remains in the zombie state
// until its parent calls wait() to find out it exited.
void
exit(void)
{
    struct proc *curproc = myproc();
    struct proc *p;
    int fd;

    if(curproc == initproc)
        panic("init exiting");

    // Close all open files.
    for(fd = 0; fd < NOFILE; fd++){

```



```

    if(curproc->ofile[fd]){
        fileclose(curproc->ofile[fd]);
        curproc->ofile[fd] = 0;
    }
}

begin_op();
input(curproc->cwd);
end_op();
curproc->cwd = 0;

acquire(&ptable.lock);

// Parent might be sleeping in wait().
wakeup1(curproc->parent);

// Pass abandoned children to init.
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->parent == curproc){
        p->parent = initproc;
        if(p->state == ZOMBIE)
            wakeup1(initproc);
    }
}

// Jump into the scheduler, never to return.
curproc->state = ZOMBIE;
sched();
panic("zombie exit");
}

// Wait for a child process to exit and return its pid.
// Return -1 if this process has no children.
int
wait(void)
{
    struct proc *p;
    int havekids, pid;
    struct proc *curproc = myproc();

    acquire(&ptable.lock);
    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->parent != curproc)
                continue;
            havekids = 1;
            if(p->state == ZOMBIE){
                // Found one.
                pid = p->pid;
                kfree(p->kstack);
                p->kstack = 0;
                freevm(p->pgdir);
                p->pid = 0;
                p->parent = 0;
                p->name[0] = 0;
                p->killed = 0;

```

```

    p->state = UNUSED;
    release(&ptable.lock);
    return pid;
}
}

// No point waiting if we don't have any children.
if(!havekids || curproc->killed){
    release(&ptable.lock);
    return -1;
}

// Wait for children to exit. (See wakeup1 call in proc_exit.)
sleep(curproc, &ptable.lock); //DOC: wait-sleep
}
}

//PAGEBREAK: 42
// Per-CPU process scheduler.
// Each CPU calls scheduler() after setting itself up.
// Scheduler never returns. It loops, doing:
//  - choose a process to run
//  - switch to start running that process
//  - eventually that process transfers control
//    via switch back to the scheduler.
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        // Enable interrupts on this processor.
        sti();

        // Loop over process table looking for process to run.
        acquire(&ptable.lock);
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)
                continue;

            // Switch to chosen process. It is the process's job
            // to release ptable.lock and then reacquire it
            // before jumping back to us.
            c->proc = p;
            switchvm(p);
            p->state = RUNNING;

            swtch(&(c->scheduler), p->context);
            switchkvm();

            // Process is done running for now.
            // It should have changed its p->state before coming back.
            c->proc = 0;
        }
        release(&ptable.lock);
    }
}

```

```

    }
}

// Enter scheduler. Must hold only ptable.lock
// and have changed proc->state. Saves and restores
// intena because intena is a property of this
// kernel thread, not this CPU. It should
// be proc->intena and proc->ncli, but that would
// break in the few places where a lock is held but
// there's no process.
void
sched(void)
{
    int intena;
    struct proc *p = myproc();

    if(!holding(&ptable.lock))
        panic("sched ptable.lock");
    if(mycpu()->ncli != 1)
        panic("sched locks");
    if(p->state == RUNNING)
        panic("sched running");
    if(readeflags() & FL_IF)
        panic("sched interruptible");
    intena = mycpu()->intena;
    swtch(&p->context, mycpu()->scheduler);
    mycpu()->intena = intena;
}

// Give up the CPU for one scheduling round.
void
yield(void)
{
    acquire(&ptable.lock); //DOC: yieldlock
    myproc()->state = RUNNABLE;
    sched();
    release(&ptable.lock);
}

// A fork child's very first scheduling by scheduler()
// will switch here. "Return" to user space.
void
forkret(void)
{
    static int first = 1;
    // Still holding ptable.lock from scheduler.
    release(&ptable.lock);

    if (first) {
        // Some initialization functions must be run in the context
        // of a regular process (e.g., they call sleep), and thus cannot
        // be run from main().
        first = 0;
        iinit(ROOTDEV);
        initlog(ROOTDEV);
    }

    // Return to "caller", actually trapret (see allocproc).

```

```
}
```

```
// Atomically release lock and sleep on chan.
```

```
// Reacquires lock when awakened.
```

```
void
```

```
sleep(void *chan, struct spinlock *lk)
```

```
{
```

```
    struct proc *p = myproc();
```

```
    if(p == 0)
```

```
        panic("sleep");
```

```
    if(lk == 0)
```

```
        panic("sleep without lk");
```

```
    // Must acquire ptable.lock in order to
```

```
    // change p->state and then call sched.
```

```
    // Once we hold ptable.lock, we can be
```

```
    // guaranteed that we won't miss any wakeup
```

```
    // (wakeup runs with ptable.lock locked),
```

```
    // so it's okay to release lk.
```

```
    if(lk != &ptable.lock){ //DOC: sleeplock0
```

```
        acquire(&ptable.lock); //DOC: sleeplock1
```

```
        release(lk);
```

```
}
```

```
    // Go to sleep.
```

```
    p->chan = chan;
```

```
    p->state = SLEEPING;
```

```
    sched();
```

```
    // Tidy up.
```

```
    p->chan = 0;
```

```
    // Reacquire original lock.
```

```
    if(lk != &ptable.lock){ //DOC: sleeplock2
```

```
        release(&ptable.lock);
```

```
        acquire(lk);
```

```
}
```

```
}
```

```
//PAGEBREAK!
```

```
// Wake up all processes sleeping on chan.
```

```
// The ptable lock must be held.
```

```
static void
```

```
wakeup1(void *chan)
```

```
{
```

```
    struct proc *p;
```

```
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
```

```
        if(p->state == SLEEPING && p->chan == chan)
```

```
            p->state = RUNNABLE;
```

```
}
```

```
// Wake up all processes sleeping on chan.
```

```
void
```

```
wakeup(void *chan)
```

```
{
```

```

    acquire(&ptable.lock);
    wakeup1(chan);
    release(&ptable.lock);
}

// Kill the process with the given pid.
// Process won't exit until it returns
// to user space (see trap in trap.c).
int
kill(int pid)
{
    struct proc *p;

    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->pid == pid){
            p->killed = 1;
            // Wake process from sleep if necessary.
            if(p->state == SLEEPING)
                p->state = RUNNABLE;
            release(&ptable.lock);
            return 0;
        }
    }
    release(&ptable.lock);
    return -1;
}

//PAGEBREAK: 36
// Print a process listing to console. For debugging.
// Runs when user types ^P on console.
// No lock to avoid wedging a stuck machine further.
void
procdump(void)
{
    static char *states[] = {
        [UNUSED]    "unused",
        [EMBRYO]    "embryo",
        [SLEEPING]  "sleep ",
        [RUNNABLE]  "runble",
        [RUNNING]   "run   ",
        [ZOMBIE]    "zombie"
    };
    int i;
    struct proc *p;
    char *state;
    uint pc[10];

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->state == UNUSED)
            continue;
        if(p->state >= 0 && p->state < NELEM(states) && states[p->state])
            state = states[p->state];
        else
            state = "???";
        cprintf("%d %s %s", p->pid, state, p->name);
        if(p->state == SLEEPING){
            getcallerpcs((uint*)p->context->ebp+2, pc);

```

```

        for(i=0; i<10 && pc[i] != 0; i++)
            cprintf(" %p", pc[i]);
    }
    cprintf("\n");
}
}

```

[proc.h]

```

#include "spinlock.h"

```

```

// Per-CPU state

```

```

struct cpu {
    uchar apicid;                // Local APIC ID
    struct context *scheduler;    // swtch() here to enter scheduler
    struct taskstate ts;         // Used by x86 to find stack for interrupt
    struct segdesc gdt[NSEGS];    // x86 global descriptor table
    volatile uint started;        // Has the CPU started?
    int ncli;                    // Depth of pushcli nesting.
    int intena;                  // Were interrupts enabled before pushcli?
    struct proc *proc;           // The process running on this cpu or null
};

```

```

extern struct cpu cpus[NCPU];
extern int ncpu;

```

```

//PAGEBREAK: 17
// Saved registers for kernel context switches.
// Don't need to save all the segment registers (%cs, etc),
// because they are constant across kernel contexts.
// Don't need to save %eax, %ecx, %edx, because the
// x86 convention is that the caller has saved them.
// Contexts are stored at the bottom of the stack they
// describe; the stack pointer is the address of the context.
// The layout of the context matches the layout of the stack in swtch.S
// at the "Switch stacks" comment. Switch doesn't save eip explicitly,
// but it is on the stack and allocproc() manipulates it.

```

```

struct context {
    uint edi;
    uint esi;
    uint ebx;
    uint ebp;
    uint eip;
};

```

```

enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

```

```

// Per-process state

```

```

struct proc {
    uint sz;                    // Size of process memory (bytes)
    pde_t* pgdir;              // Page table
    char *kstack;              // Bottom of kernel stack for this process
    enum procstate state;      // Process state
    int pid;                   // Process ID
    struct proc *parent;       // Parent process
    struct trapframe *tf;      // Trap frame for current syscall
    struct context *context;    // swtch() here to run process

```

```

void *chan:                // If non-zero, sleeping on chan
int killed;                // If non-zero, have been killed
struct file *ofile[NOFILE]; // Open files
struct inode *cwd;         // Current directory
char name[16];             // Process name (debugging)
};

// Process memory is laid out contiguously, low addresses first:
//  text
//  original data and bss
//  fixed-size stack
//  expandable heap

// ptable을 sysproc.c에서 쓰기 위해 extern으로 선언
// 이때 익명 struct를 쓰지 않고, 공용 타입을 만들어서 extern/define을 맞춰줌

```

```

struct ptable_t {
    struct spinlock lock;
    struct proc proc[NPROC];
};

```

```

extern struct ptable_t ptable;

```

```

//extern struct{
//    struct spinlock lock;
//    struct proc proc[NPROC];
//} ptable;

```

[syscall.c]

```

#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "x86.h"
#include "syscall.h"

```

```

// User code makes a system call with INT T_SYSCALL.
// System call number in %eax.
// Arguments on the stack, from the user call to the C
// library system call function. The saved user %esp points
// to a saved program counter, and then the first argument.

```

```

// Fetch the int at addr from the current process.

```

```

int
fetchint(uint addr, int *ip)
{
    struct proc *curproc = myproc();

    if(addr >= curproc->sz || addr+4 > curproc->sz)
        return -1;
    *ip = *(int*)(addr);
    return 0;
}

```

```

// Fetch the nul-terminated string at addr from the current process.

```



```

// Doesn't actually copy the string - just sets *pp to point at it.
// Returns length of string, not including nul.
int
fetchstr(uint addr, char **pp)
{
    char *s, *ep;
    struct proc *curproc = myproc();

    if(addr >= curproc->sz)
        return -1;
    *pp = (char*)addr;
    ep = (char*)curproc->sz;
    for(s = *pp; s < ep; s++){
        if(*s == 0)
            return s - *pp;
    }
    return -1;
}

// Fetch the nth 32-bit system call argument.
int
argint(int n, int *ip)
{
    return fetchint((myproc()->tf->esp) + 4 + 4*n, ip);
}

// Fetch the nth word-sized system call argument as a pointer
// to a block of memory of size bytes. Check that the pointer
// lies within the process address space.
int
argptr(int n, char **pp, int size)
{
    int i;
    struct proc *curproc = myproc();

    if(argint(n, &i) < 0)
        return -1;
    if(size < 0 || (uint)i >= curproc->sz || (uint)i+size > curproc->sz)
        return -1;
    *pp = (char*)i;
    return 0;
}

// Fetch the nth word-sized system call argument as a string pointer.
// Check that the pointer is valid and the string is nul-terminated.
// (There is no shared writable memory, so the string can't change
// between this check and being used by the kernel.)
int
argstr(int n, char **pp)
{
    int addr;
    if(argint(n, &addr) < 0)
        return -1;
    return fetchstr(addr, pp);
}

extern int sys_chdir(void);
extern int sys_close(void);

```

```

extern int sys_dup(void);
extern int sys_exec(void);
extern int sys_exit(void);
extern int sys_fork(void);
extern int sys_fstat(void);
extern int sys_getpid(void);
extern int sys_kill(void);
extern int sys_link(void);
extern int sys_mkdir(void);
extern int sys_mknod(void);
extern int sys_open(void);
extern int sys_pipe(void);
extern int sys_read(void);
extern int sys_sbrk(void);
extern int sys_sleep(void);
extern int sys_unlink(void);
extern int sys_wait(void);
extern int sys_write(void);
extern int sys_uptime(void);
extern int sys_hello_number(void); // sys_hello_number 시스템콜 추가
extern int sys_get_procinfo(void); // sys_get_procinfo 시스템콜 추가

static int (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]    sys_fstat,
[SYS_chdir]    sys_chdir,
[SYS_dup]     sys_dup,
[SYS_getpid]   sys_getpid,
[SYS_sbrk]     sys_sbrk,
[SYS_sleep]    sys_sleep,
[SYS_uptime]   sys_uptime,
[SYS_open]     sys_open,
[SYS_write]    sys_write,
[SYS_mknod]    sys_mknod,
[SYS_unlink]   sys_unlink,
[SYS_link]     sys_link,
[SYS_mkdir]    sys_mkdir,
[SYS_close]    sys_close,
[SYS_hello_number] sys_hello_number, // sys_hello_number 시스템콜 추가
[SYS_get_procinfo] sys_get_procinfo, // sys_get_procinfo 시스템콜 추가
};

void
syscall(void)
{
    int num;
    struct proc *curproc = myproc();

    num = curproc->tf->eax;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        curproc->tf->eax = syscalls[num]();
    } else {

```

```

        cprintf("%d %s: unknown sys call %d\n",
                curproc->pid, curproc->name, num);
        curproc->tf->eax = -1;
    }
}

```

[syscall.h]

```

// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat    8
#define SYS_chdir    9
#define SYS_dup     10
#define SYS_getpid  11
#define SYS_sbrk    12
#define SYS_sleep   13
#define SYS_uptime  14
#define SYS_open    15
#define SYS_write   16
#define SYS_mknod   17
#define SYS_unlink  18
#define SYS_link    19
#define SYS_mkdir   20
#define SYS_close   21
#define SYS_hello_number 22 // 22번 시스템콜을 hello_number로 추가
#define SYS_get_procinfo 23 // 23번 시스템콜을 get_procinfo로 추가

```

[sysproc.c]

```

#include "types.h"
#include "x86.h"
#include "defs.h"
#include "date.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"

int
sys_fork(void)
{
    return fork();
}

int
sys_exit(void)
{
    exit();
    return 0; // not reached
}

int
sys_wait(void)
{

```

```

    return wait();
}

int
sys_kill(void)
{
    int pid;

    if(argint(0, &pid) < 0)
        return -1;
    return kill(pid);
}

int
sys_getpid(void)
{
    return myproc()->pid;
}

int
sys_sbrk(void)
{
    int addr;
    int n;

    if(argint(0, &n) < 0)
        return -1;
    addr = myproc()->sz;
    if(growproc(n) < 0)
        return -1;
    return addr;
}

int
sys_sleep(void)
{
    int n;
    uint ticks0;

    if(argint(0, &n) < 0)
        return -1;
    acquire(&tickslock);
    ticks0 = ticks;
    while(ticks - ticks0 < n){
        if(myproc()->killed){
            release(&tickslock);
            return -1;
        }
        sleep(&ticks, &tickslock);
    }
    release(&tickslock);
    return 0;
}

// return how many clock tick interrupts have occurred
// since start.
int
sys_uptime(void)

```

```

{
    uint xticks;

    acquire(&tickslock);
    xticks = ticks;
    release(&tickslock);
    return xticks;
}

// xv6 시스템 콜 핸들러 규칙: sys_를 앞에 붙인다
int
sys_hello_number(void){
    int n;    // 시스템 콜 인자로 받을 정수를 저장할 변수
    // argint: xv6 헬퍼 함수: 시스템 콜 인자에서 정수 하나를 꺼내서 두 번째 매개변수 n에 저장
    if(argint(0, &n) < 0)    // argint가 음수를 반환하면 실패
        return -1;
    cprintf("Hello, xv6! Your number is %d\n", n);    // xv6 커널에서 사용하는 printf 버전 = cprintf
    return n * 2;    // 커널에서 n*2로 연산하고, 이 결과를 유저 프로그램에 리턴 값으로
전달한다
}

// 커널 내부 구조체(struct proc)와 사용자 공간 인터페이스를 분리하기 위해 struct k_procinfo를 따로 만듦
// 즉 커널이 쓰는 민감한 정보들은 빼고, 사용자 공간으로 보낼 일부 정보만 추려낸 구조체를 정의
struct k_procinfo{
    int pid;
    int ppid;
    int state;
    uint sz;
    char name[16];
};

// pid
int sys_get_procinfo(void){
    int pid;
    char *uaddr;    // 유저 버퍼 시작 주소
    struct proc *p, *t;
    struct k_procinfo kinfo;

    // 시스템콜의 첫 번째 인자(0)를 int로 꺼내서 pid 변수에 저장
    // 사용자가 넘긴 pid값을 커널 변수 pid로 안전하게 가져온다
    if(argint(0, &pid) < 0)
        return -1;
    // 시스템콜의 두 번째 인자(1)를 포인터로 받아온다
    // 즉, 유저가 넘긴 포인터가 가리키는 주소(struct k_procinfo *uinfo의 주소)가 가리키는 주소를
    // uaddr에 저장한다
    if(argptr(1, &uaddr, sizeof(struct k_procinfo)) < 0)
        return -1;

    // 프로세스 테이블(ptable)에 접근할 때 동시성 문제를 막기 위해 락을 잡음
    // 락 관련 문제 발생
    acquire(&ptable.lock);

    // pid <= 0 : 호출한 자기 자신 정보 조회
    if(pid <= 0)
        t = myproc();    // 현재 실행 중인 프로세스를 가져오는 함수
    // pid > 0 : 해당 PID 프로세스의 정보를 조회
    else{
        t = 0;
    }
}

```

```

        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
            if(p->pid == pid) {
                t = p;
                break;
            }
    }

    // 해당 프로세스(t)가 없거나 UNUSED 상태면 에러 반환
    if(t == 0 || (t->state == UNUSED)) {
        release(&ptable.lock);
        return -1;
    }

    // t에서 필요한 값 추출
    // 입력받은 PID의 프로세스 정보를 kinfo 구조체에 저장한다
    kinfo.pid = t->pid; // 프로세스 PID
    kinfo.ppid = t->parent ? t->parent->pid : 0; // 부모 프로세스 PID(없으면 0)
    kinfo.state = t->state; // 프로세스 상태
    kinfo.sz = t->sz; // 메모리 크기(bytes)
    safestrcpy(kinfo.name, t->name, sizeof(kinfo.name)); // 프로세스 이름

    // copyout/return을 하려면 락을 풀어야 한다
    release(&ptable.lock);

    // 유저 공간으로 복사
    // copyout()을 이용해 커널 공간의 kinfo를 사용자 버퍼(uaddr)로 복사한다.
    if(copyout(myproc()->pgdir, (uint)uaddr, (void*)&kinfo, sizeof(kinfo)) < 0)
        return -1;

    return 0;
}

```

[user.h]

```

// user.h -> 유저용 구조체 + 프로토타입
// 즉 사용자 프로그램이 커널의 시스템 콜을 쓸 수 있도록 선언해 두는 헤더(여기에 추가해야 커널 내부 시스템콜을 사용자
// 프로그램에서 함수처럼 쓸 수 있게 된다)
struct stat;
struct rtcdate;

// system calls
int fork(void);
int exit(void) __attribute__((noreturn));
int wait(void);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);

```

```

char* sbrk(int);
int sleep(int);
int uptime(void);
int hello_number(int n);    // 사용자 공간에서 hello_number()를 사용하기 위해 프로토타입 선언

// ulib.c
int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void *memmove(void*, const void*, int);
char* strchr(const char*, char c);
int strcmp(const char*, const char*);
void printf(int, const char*, ...);
char* gets(char*, int max);
uint strlen(const char*);
void* memset(void*, int, uint);
void* malloc(uint);
void free(void*);
int atoi(const char*);

// 유저용 선언
// 커널에서 프로세스 정보를 조회해서 사용자 프로그램으로 넘겨줄텐데, 이때 사용자 프로그램에서도 같은 자료형 정의를 알아야
// 복사 받을 수 있다.
// 따라서 아래 구조체 정의도 꼭 추가해야 한다
struct procinfo{
    int pid;        // 대상 PID
    int ppid;       // 부모 PID
    int state;      // enum procstate 값
    uint sz;        // 메모리 크기(바이트)
    char name[16];  // 프로세스 이름
};
int get_procinfo(int pid, struct procinfo *uinfo);    // 사용자 공간에서 get_procinfo()를 사용하기 위해 프로토타입 선언
-----
[usys.S]
// usys.S: 유저 공간에서 시스템콜을 함수처럼 호출할 수 있도록 아주 얇은 래퍼를 만들어 준다(wrapper)
// hello_number(7) 사용자 함수 호출 -> usys.S(어셈블리 코드)에서 int $T_SYSCALL 트랩 -> 커널 진입 ->
sys_hello_number() 실행

#include "syscall.h" // 시스템콜 번호가 정의되어 있음
#include "traps.h"

#define SYSCALL(name) \        // 어셈블리 코드 자동 생성
.globl name; \                // name에 해당하는 심볼을 전역으로 노출
name: \                        // 함수 진입점
    movl $SYS_ ## name, %eax; \    // 시스템콜 번호를 eax 레지스터에 넣음
    int $T_SYSCALL; \            // 소프트웨어 인터럽트 발생. CPU는 커널 모드로
    전환(alltraps -> syscall() 처리 루틴 실행)
    ret                        // 시스템콜 실행이 끝나고 다시 사용자
    공간으로 돌아오면 리턴

SYSCALL(fork)
SYSCALL(exit)
SYSCALL(wait)
SYSCALL(pipe)
SYSCALL(read)
SYSCALL(write)
SYSCALL(close)
SYSCALL(kill)
SYSCALL(exec)

```

```

SYSCALL(open)
SYSCALL(mknod)
SYSCALL(unlink)
SYSCALL(fstat)
SYSCALL(link)
SYSCALL(mkdir)
SYSCALL(chdir)
SYSCALL(dup)
SYSCALL(getpid)
SYSCALL(sbrk)
SYSCALL(sleep)
SYSCALL(uptime)
SYSCALL(hello_number) // hello_number 시스템 콜에 대한 어셈블리 래퍼
SYSCALL(get_procinfo) // get_procinfo 시스템 콜에 대한 어셈블리 래퍼

```

[spinlock.h]

```

#ifndef SPINLOCK_H
#define SPINLOCK_H

#include "types.h"

struct cpu; // 전방선언

// Mutual exclusion lock.
struct spinlock {
    uint locked; // 현재 이 락이 잡혀있으면 1, 아니면 0

    char *name; // 디버깅용 문자열 이름. 예를 들어 "ptable" 같은 이름을 붙여두면, 프로세스 테이블 락인지 바로 알 수 있음
    struct cpu *cpu; // 현재 이 락을 들고 있는 CPU를 가리킨다.
    uint pcs[10]; // 락을 잡았을 때의 호출 스택(program counter)들을 저장한다
};

#endif

```

[Makefile]

```

OBJS = \
    bio.o\
    console.o\
    exec.o\
    file.o\
    fs.o\
    ide.o\
    ioapic.o\
    kalloc.o\
    kbd.o\
    lapic.o\
    log.o\
    main.o\
    mp.o\
    picirq.o\
    pipe.o\
    proc.o\
    sleeplock.o\
    spinlock.o\
    string.o\
    swtch.o\

```



```

syscall.o\
sysfile.o\
sysproc.o\
trapasm.o\
trap.o\
uart.o\
vectors.o\
vm.o\

# Cross-compiling (e.g., on Mac OS X)
# TOOLPREFIX = i386-jos-elf

# Using native tools (e.g., on X86 Linux)
#TOOLPREFIX =

# Try to infer the correct TOOLPREFIX if not set
ifndef TOOLPREFIX
TOOLPREFIX := $(shell if i386-jos-elf-objdump -i 2>&1 | grep '^elf32-i386$$' >/dev/null 2>&1; \
    then echo 'i386-jos-elf-'; \
    elif objdump -i 2>&1 | grep 'elf32-i386' >/dev/null 2>&1; \
    then echo ''; \
    else echo "***" 1>&2; \
    echo "*** Error: Couldn't find an i386-*-elf version of GCC/binutils." 1>&2; \
    echo "*** Is the directory with i386-jos-elf-gcc in your PATH?" 1>&2; \
    echo "*** If your i386-*-elf toolchain is installed with a command" 1>&2; \
    echo "*** prefix other than 'i386-jos-elf-', set your TOOLPREFIX" 1>&2; \
    echo "*** environment variable to that prefix and run 'make' again." 1>&2; \
    echo "*** To turn off this error, run 'gmake TOOLPREFIX= ...'." 1>&2; \
    echo "***" 1>&2; exit 1; fi)
endif

# If the makefile can't find QEMU, specify its path here
# QEMU = qemu-system-i386

# Try to infer the correct QEMU
ifndef QEMU
QEMU = $(shell if which qemu > /dev/null; \
    then echo qemu; exit; \
    elif which qemu-system-i386 > /dev/null; \
    then echo qemu-system-i386; exit; \
    elif which qemu-system-x86_64 > /dev/null; \
    then echo qemu-system-x86_64; exit; \
    else \
    qemu=/Applications/Q.app/Contents/MacOS/i386-softmmu.app/Contents/MacOS/i386-softmmu; \
    if test -x $$qemu; then echo $$qemu; exit; fi; \
    echo "***" 1>&2; \
    echo "*** Error: Couldn't find a working QEMU executable." 1>&2; \
    echo "*** Is the directory containing the qemu binary in your PATH" 1>&2; \
    echo "*** or have you tried setting the QEMU variable in Makefile?" 1>&2; \
    echo "***" 1>&2; exit 1)
endif

CC = $(TOOLPREFIX)gcc
AS = $(TOOLPREFIX)gas
LD = $(TOOLPREFIX)ld
OBJCOPY = $(TOOLPREFIX)objcopy
OBJDUMP = $(TOOLPREFIX)objdump
CFLAGS = -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror

```

```

-fno-omit-frame-pointer
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)
ASFLAGS = -m32 -gdwarf-2 -Wa,-divide
# FreeBSD ld wants ``elf_i386_fbsd''
LDLFLAGS += -m $(shell $(LD) -V | grep elf_i386 2>/dev/null | head -n 1)

# Disable PIE when possible (for Ubuntu 16.10 toolchain)
ifneq ($(shell $(CC) -dumpspeaks 2>/dev/null | grep -e '^[^f]no-pie'),)
CFLAGS += -fno-pie -no-pie
endif
ifneq ($(shell $(CC) -dumpspeaks 2>/dev/null | grep -e '^[^f]nopie'),)
CFLAGS += -fno-pie -nopie
endif

xv6.img: bootblock kernel
    dd if=/dev/zero of=xv6.img count=10000
    dd if=bootblock of=xv6.img conv=notrunc
    dd if=kernel of=xv6.img seek=1 conv=notrunc

xv6memfs.img: bootblock kernelmemfs
    dd if=/dev/zero of=xv6memfs.img count=10000
    dd if=bootblock of=xv6memfs.img conv=notrunc
    dd if=kernelmemfs of=xv6memfs.img seek=1 conv=notrunc

bootblock: bootasm.S bootmain.c
    $(CC) $(CFLAGS) -fno-pic -O -nostdinc -I. -c bootmain.c
    $(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c bootasm.S
    $(LD) $(LDLFLAGS) -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
    $(OBJDUMP) -S bootblock.o > bootblock.asm
    $(OBJCOPY) -S -O binary -j .text bootblock.o bootblock
    ./sign.pl bootblock

entryother: entryother.S
    $(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c entryother.S
    $(LD) $(LDLFLAGS) -N -e start -Ttext 0x7000 -o bootblockother.o entryother.o
    $(OBJCOPY) -S -O binary -j .text bootblockother.o entryother
    $(OBJDUMP) -S bootblockother.o > entryother.asm

initcode: initcode.S
    $(CC) $(CFLAGS) -nostdinc -I. -c initcode.S
    $(LD) $(LDLFLAGS) -N -e start -Ttext 0 -o initcode.out initcode.o
    $(OBJCOPY) -S -O binary initcode.out initcode
    $(OBJDUMP) -S initcode.o > initcode.asm

kernel: $(OBJS) entry.o entryother initcode kernel.ld
    $(LD) $(LDLFLAGS) -T kernel.ld -o kernel entry.o $(OBJS) -b binary initcode entryother
    $(OBJDUMP) -S kernel > kernel.asm
    $(OBJDUMP) -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernel.sym

# kernelmemfs is a copy of kernel that maintains the
# disk image in memory instead of writing to a disk.
# This is not so useful for testing persistent storage or
# exploring disk buffering implementations, but it is
# great for testing the kernel on real hardware without
# needing a scratch disk.
MEMFSOBS = $(filter-out ide.o,$(OBJS)) memide.o
kernelmemfs: $(MEMFSOBS) entry.o entryother initcode kernel.ld fs.img
    $(LD) $(LDLFLAGS) -T kernel.ld -o kernelmemfs entry.o $(MEMFSOBS) -b binary initcode entryother fs.img

```

```

$(OBJDUMP) -S kernelmemfs > kernelmemfs.asm
$(OBJDUMP) -t kernelmemfs | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernelmemfs.sym

tags: $(OBJS) entryother.S _init
      etags *.S *.c

vectors.S: vectors.pl
      ./vectors.pl > vectors.S

ULIB = ulib.o usys.o printf.o umalloc.o

_%: %.o $(ULIB)
      $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $@ $^
      $(OBJDUMP) -S $@ > $.asm
      $(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $.sym

_forktest: forktest.o $(ULIB)
      # forktest has less library code linked in - needs to be small
      # in order to be able to max out the proc table.
      $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o _forktest forktest.o ulib.o usys.o
      $(OBJDUMP) -S _forktest > forktest.asm

mkfs: mkfs.c fs.h
      gcc -Werror -Wall -o mkfs mkfs.c

# Prevent deletion of intermediate files, e.g. cat.o, after first build, so
# that disk image changes after first build are persistent until clean. More
# details:
# http://www.gnu.org/software/make/manual/html_node/Chained-Rules.html
.PRECIOUS: %.o

# UPROGS에 _helloxv6, _psinfo를 추가해야 xv6 유저 프로그램으로 컴파일되고, xv6 파일시스템 이미지에 포함된다
UPROGS=\
_cat\
_echo\
_forktest\
_grep\
_init\
_kill\
_ln\
_ls\
_mkdir\
_rm\
_sh\
_stressfs\
_usertests\
_wc\
_zombie\
_helloxv6\
_psinfo\

fs.img: mkfs README $(UPROGS)
      ./mkfs fs.img README $(UPROGS)

-include *.d

clean:
      rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \

```

```

*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
$(UPROGS)

# make a printout
FILES = $(shell grep -v '^\\#' runoff.list)
PRINT = runoff.list runoff.spec README toc.hdr toc.ftr $(FILES)

xv6.pdf: $(PRINT)
    ./runoff
    ls -l xv6.pdf

print: xv6.pdf

# run in emulators

bochs : fs.img xv6.img
    if [ ! -e .bochsrc ]; then ln -s dot-bochsrc .bochsrc; fi
    bochs -q

# try to generate a unique GDB port
GDBPORT = $(shell expr `id -u` % 5000 + 25000)
# QEMU's gdb stub command line changed in 0.11
QEMUGDB = $(shell if $(QEMU) -help | grep -q '^~gdb'; \
    then echo "-gdb tcp::$(GDBPORT)"; \
    else echo "-s -p $(GDBPORT)"; fi)
ifndef CPUS
CPUS := 2
endif
QEMUOPTS = -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp
$(CPUS) -m 512 $(QEMUEXTRA)

qemu: fs.img xv6.img
    $(QEMU) -serial mon:stdio $(QEMUOPTS)

qemu-memfs: xv6memfs.img
    $(QEMU) -drive file=xv6memfs.img,index=0,media=disk,format=raw -smp $(CPUS) -m 256

qemu-nox: fs.img xv6.img
    $(QEMU) -nographic $(QEMUOPTS)

.gdbinit: .gdbinit.tmpl
    sed "s/localhost:1234/localhost:$(GDBPORT)/" < $^ > $@

qemu-gdb: fs.img xv6.img .gdbinit
    @echo "*** Now run 'gdb'." 1>&2
    $(QEMU) -serial mon:stdio $(QEMUOPTS) -S $(QEMUGDB)

qemu-nox-gdb: fs.img xv6.img .gdbinit
    @echo "*** Now run 'gdb'." 1>&2
    $(QEMU) -nographic $(QEMUOPTS) -S $(QEMUGDB)

# CUT HERE

# prepare dist for students
# after running make dist, probably want to
# rename it to rev0 or rev1 or so on and then
# check in that version.

```

```

EXTRA=\
    mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill.c\
    ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c zombie.c\
    printf.c umalloc.c\
    README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
    .gdbinit.tmpl gdbutil\

dist:
    rm -rf dist
    mkdir dist
    for i in $(FILES); \
    do \
        grep -v PAGEBREAK $$i >dist/$$i; \
    done
    sed '/CUT HERE/,,$$d' Makefile >dist/Makefile
    echo >dist/runoff.spec
    cp $(EXTRA) dist

dist-test:
    rm -rf dist
    make dist
    rm -rf dist-test
    mkdir dist-test
    cp dist/* dist-test
    cd dist-test; $(MAKE) print
    cd dist-test; $(MAKE) bochs || true
    cd dist-test; $(MAKE) qemu

# update this rule (change rev#) when it is time to
# make a new revision.
tar:
    rm -rf /tmp/xv6
    mkdir -p /tmp/xv6
    cp dist/* dist/.gdbinit.tmpl /tmp/xv6
    (cd /tmp; tar cf - xv6) | gzip >xv6-rev10.tar.gz # the next one will be 10 (9/17)

```

```

.PHONY: dist-test dist

```

[helloxv6.c]

```

// helloxv6.c
// user.h 안에 int hello_number(int n); 함수 원형이 들어 있어서 호출 가능해짐

```

```

#include "types.h"
#include "stat.h"
#include "user.h"

```

```

int main(int argc, char *argv[]){

    // 시스템콜 호출: 커널의 sys_hello_number 실행.
    int res = hello_number(5);
    printf(1, "hello_number(5) returned %d\n", res);

    // 커널이 인자로 음수를 받을 때 테스트
    int res2 = hello_number(-7);
    printf(1, "hello_number(-7) returned %d\n", res2);

    // 커널이 인자로 0을 받을 때 테스트

```

```

int res3 = hello_number(0);
printf(1, "hello_number(0) returned %d\n", res3);

// 큰 값을 받을 때 테스트
int res4 = hello_number(100000);
printf(1, "hello_number(100000) returned %d\n", res4);

exit();
}
-----
[psinfo.c]
//psinfo.c
#include "types.h"
#include "stat.h"
#include "user.h"

static char* s2str(int s){
    // 상태 코드를 문자열로 변환한다
    // 커널이 enum procstate{ } 순서(정수 0~5)에 맞춰 문자열로 바꾼다.
    switch(s){
        case 0:
            return "UNUSED";
        case 1:
            return "EMBRYO";
        case 2:
            return "SLEEPING";
        case 3:
            return "RUNNABLE";
        case 4:
            return "RUNNING";
        case 5:
            return "ZOMBIE";
    }
    return "UNKNOWN";
}

int main(int argc, char *argv[])
{
    struct procinfo info;
    int pid = (argc >= 2) ? atoi(argv[1]) : 0;    // pid=0이면 자기 자신

    // info 구조체에 아래의 5가지 정보를 저장한다
    // 이때 sysproc.c에서 정의했던 get_procinfo() 함수를 사용하면 된다.
    if(get_procinfo(pid, &info) < 0)
    {
        // 시스템 콜 호출이 실패하면 에러 메시지 후 종료
        // printf의 첫 인자 1은 xv6에서 stdout 파일 디스크립터이다.
        printf(1, "psinfo: failed (pid=%d)\n", pid);
        exit();
    }
    // 지정한 PID의 정보를 한 줄로 출력한다
    printf(1, "PID=%d PPID=%d STATE=%s SZ=%d NAME=%s\n", info.pid, info.ppid, s2str(info.state),
info.sz, info.name);
    exit();
}

```